

Incremental Data Loading with Azure Databricks

CDC with Spark, Databricks SQL, DLT, Autoloader

...

Dustin Vannoy

Specialist - Field Engineering @ Databricks

Agenda

Incremental ETL

Databricks Lakehouse short review

Common Azure sources and sinks

Spark Structured Streaming (+Demo)

Autoloader (+Demo)

Delta Live Tables (+Demo)

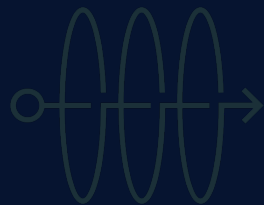
Incremental ETL

Streaming Data

Continuously generated and unbounded data



DB Change
Data Feeds



Clickstreams



Machine &
Application
Logs



Application
Events



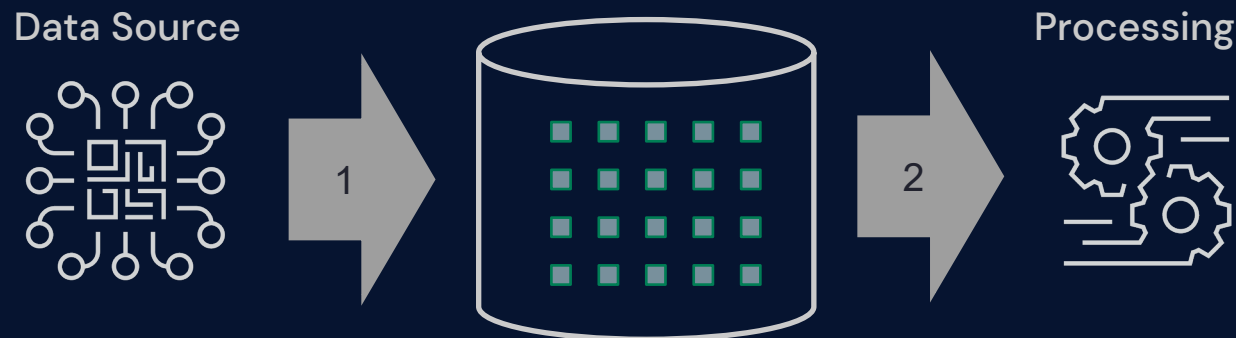
Mobile & IoT
Data



The vast majority of the data in the world is streaming data!

Stream Processing

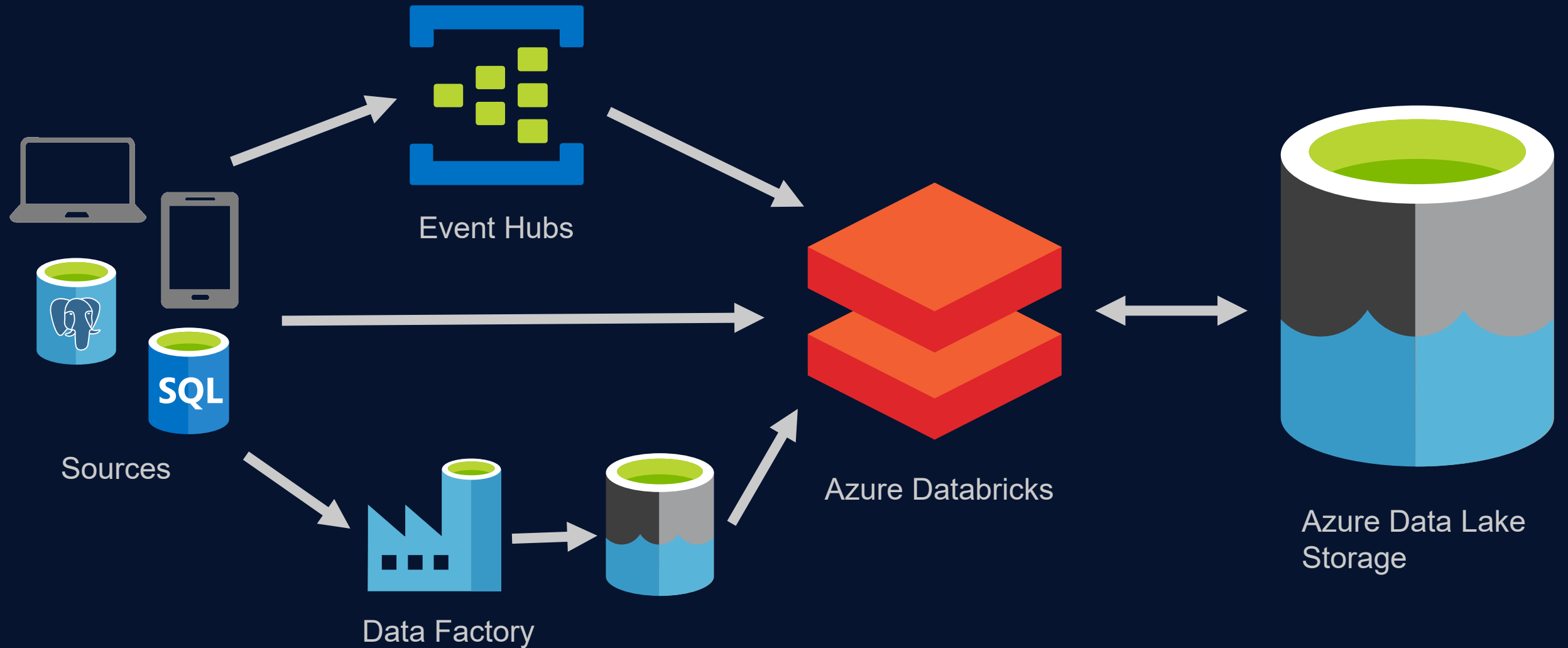
Traditional Processing is
one-off and bounded



Stream Processing is
continuous and unbounded



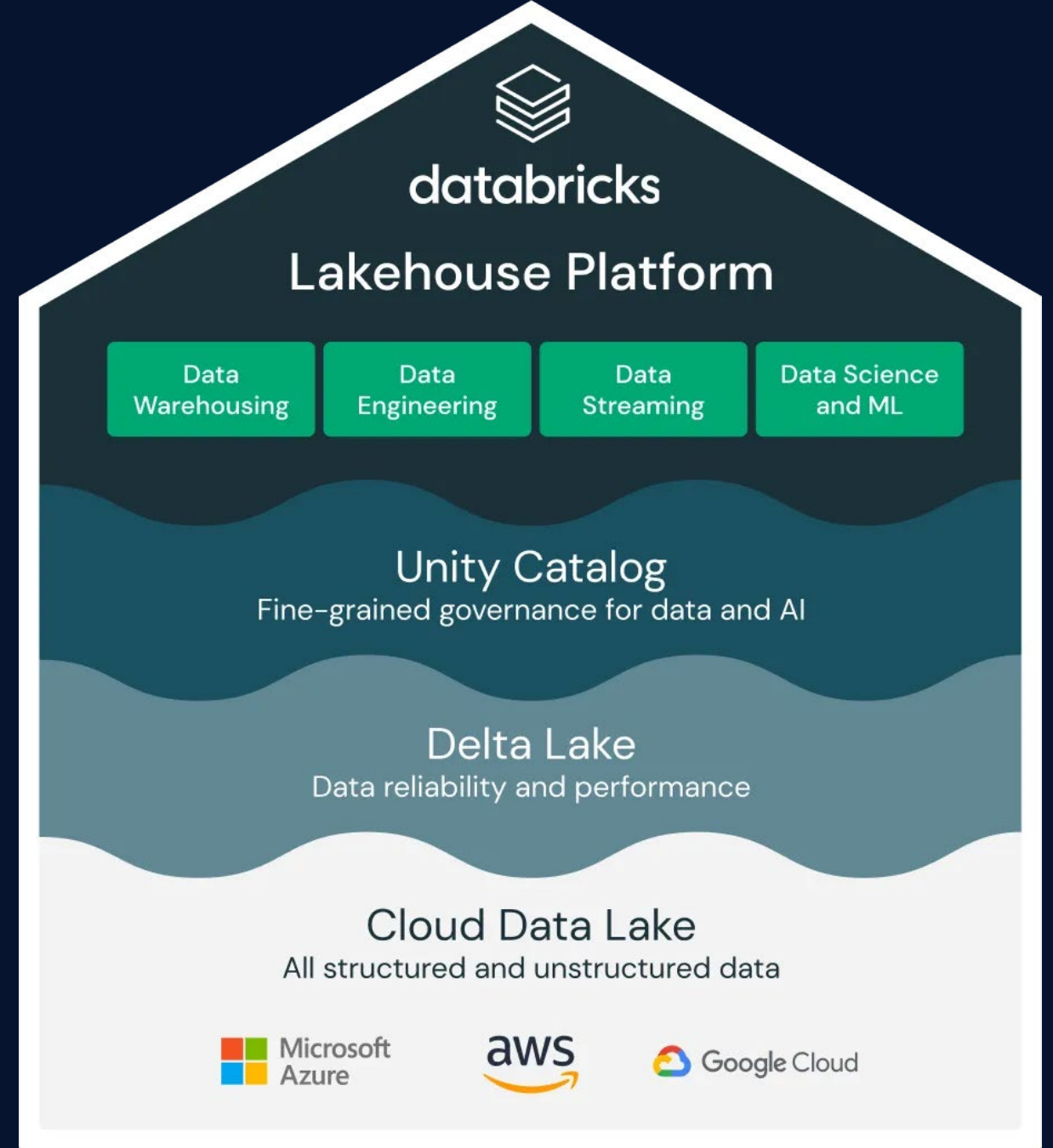
Databricks Data Lake - Ingest Options



Databricks Lakehouse Overview

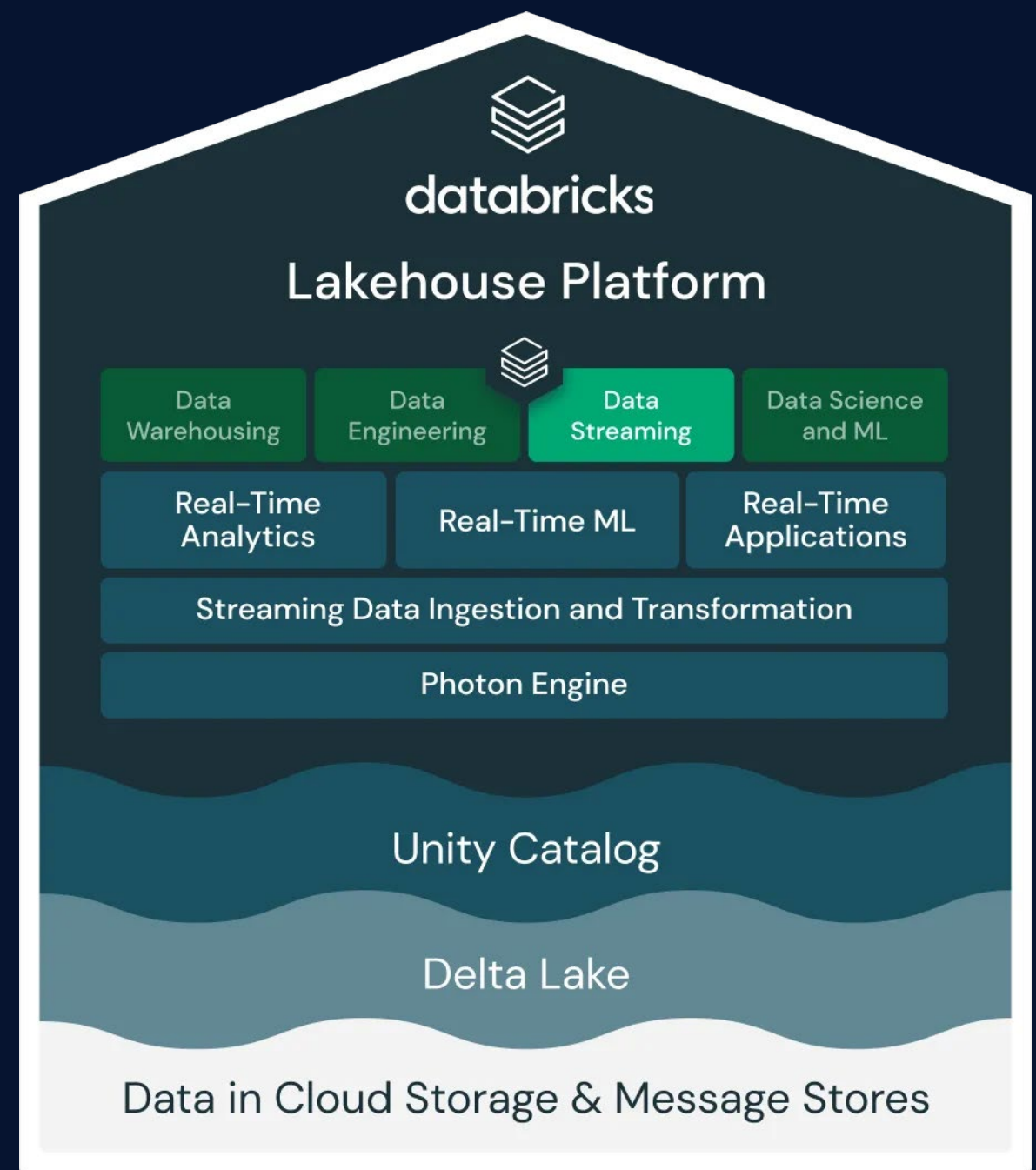
Databricks Lakehouse

- Datasets saved in cloud storage
- Access same open-format for multiples uses
- Ensure reliability and governance



Data Ingestion on Lakehouse

- Built-in tools for streaming data ingestion
- Photon engine speeds up many real-world queries
- Delta Lake enables file streaming



Azure Sources and Sinks

Azure Database

Azure SQL

PostgreSQL

MySQL

MariaDB



Azure Database

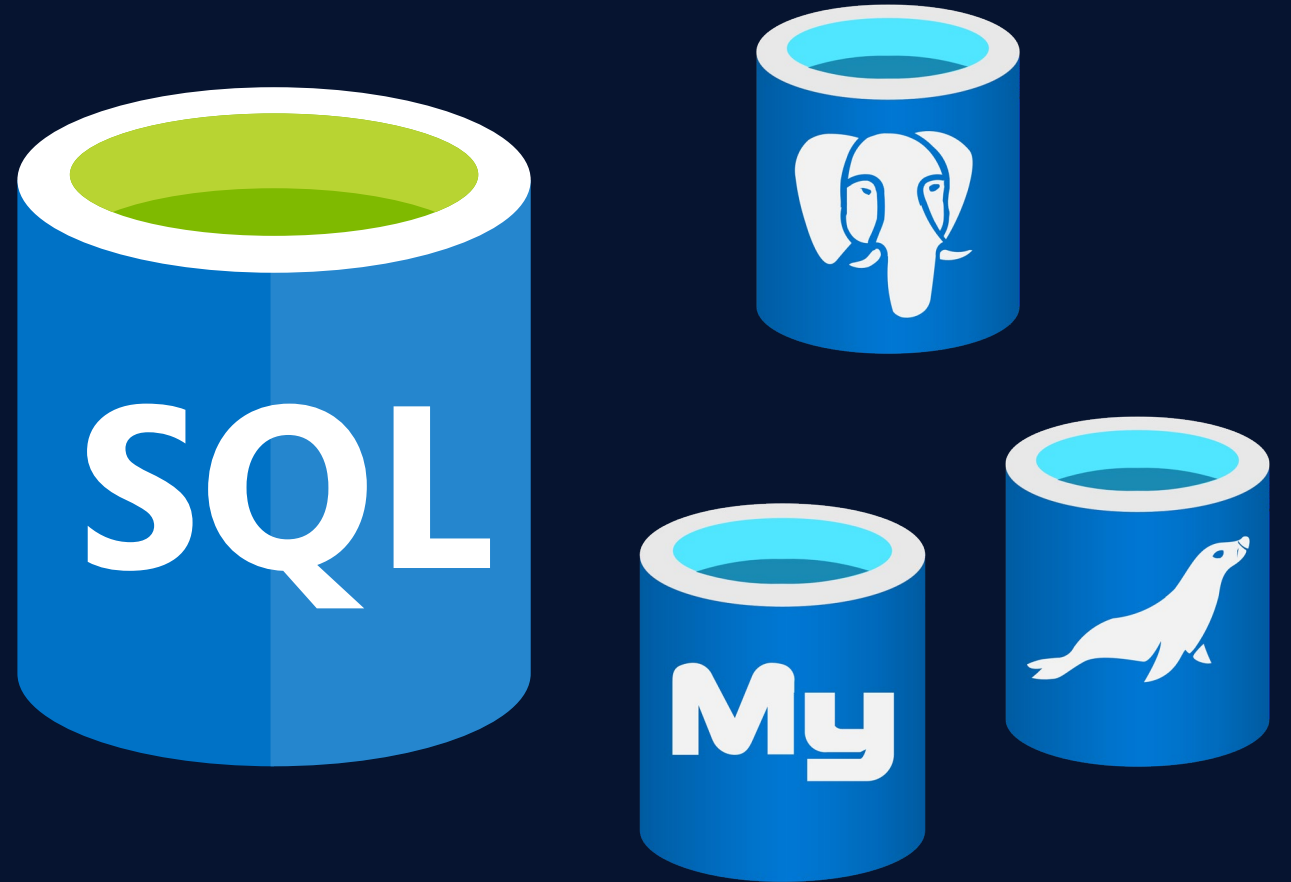
Options:

JDBC

sql-spark-connector

Fivetran / Qlik Replicate

ADF -> ADLS



Autoloader (+Demo)

Why Autoloader?

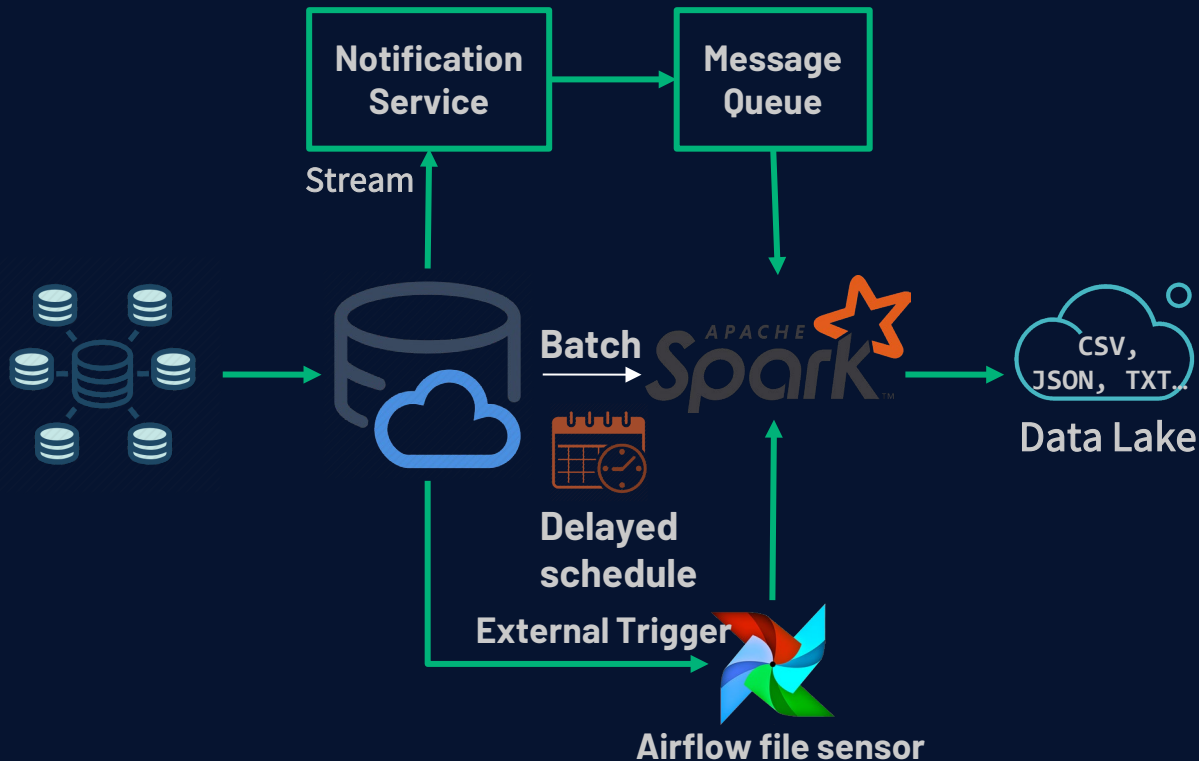
Simplify incremental
load from cloud
storage (ADLS)

- Consistent approach to read from cloud object storage
- Track state of what has been loaded
- Remove complexity with custom library

Databricks Ingest: Auto Loader

Load new data easily and efficiently as it arrives in cloud storage

Before



Gets too complicated for multiple jobs

After



- Pipe data from cloud storage into your data lake with Delta Lake as it arrives
- “Set and forget” model eliminates complex setup

[Launch Blog Post](#)

Auto Loader Syntax

```
df = spark.readStream.format("cloudFiles")  
    .option("cloudFiles.includeExistingFiles", false)  
    .option("cloudFiles.format", "json")  
    .schema(...)  
    .load("/Volume/demo/input_path")
```

Auto Loader Benefits

- **Incremental load without custom state management**
 - Incrementally process new files as they land on cloud object storage
 - State managed automatically, behind the scenes
- **Scalable file tracking**
 - Efficiently track new file arrival using Event Grid subscription and Azure Queue Storage
 - Scalable to handle millions of files in a directory
 - Less cost for file listing operations
- **Easy to use**
 - Automatically sets up AQS
 - Simple options for schema inference and file format

Delta Live Tables (+Demo)

Delta Live Tables

```
CREATE STREAMING TABLE raw_data
AS SELECT *
FROM cloud_files ("/raw_data", "json")
```

```
CREATE LIVE TABLE clean_data
AS SELECT ...
FROM LIVE .raw_data
```



Accelerate ETL development

Declare **SQL or Python** and DLT automatically orchestrates the DAG, handles retries, changing data



Automatically manage your infrastructure

Automates complex tedious activities like **recovery**, **auto-scaling**, and **performance optimization**



Ensure high data quality

Deliver reliable data with built-in **quality controls**, **testing**, **monitoring**, and **enforcement**



Unify batch and streaming

Get the simplicity of SQL with freshness of streaming with one **unified API**

Transform streaming data with declarative SQL & Python APIs for easy development

Source

```
/* Create a temp view on the accounts table */  
CREATE STREAMING VIEW account_raw AS  
SELECT * FROM cloud_files("/data", "csv");
```

Bronze

```
/* Stage 1: Bronze Table drop invalid rows */  
CREATE STREAMING TABLE account_bronze AS  
COMMENT "Bronze table with valid account ids"  
SELECT * FROM account_raw ...
```

Silver

```
/* Stage 2: Send rows to Silver, run validation rules */  
CREATE STREAMING TABLE account_silver AS  
COMMENT "Silver Accounts table with validation checks"  
SELECT * FROM account_bronze ...
```

Gold

Use intent-driven declarative development to abstract away the “**how**” and define “**what**” to solve

Automatically generate **lineage** based on table dependencies across the data pipeline

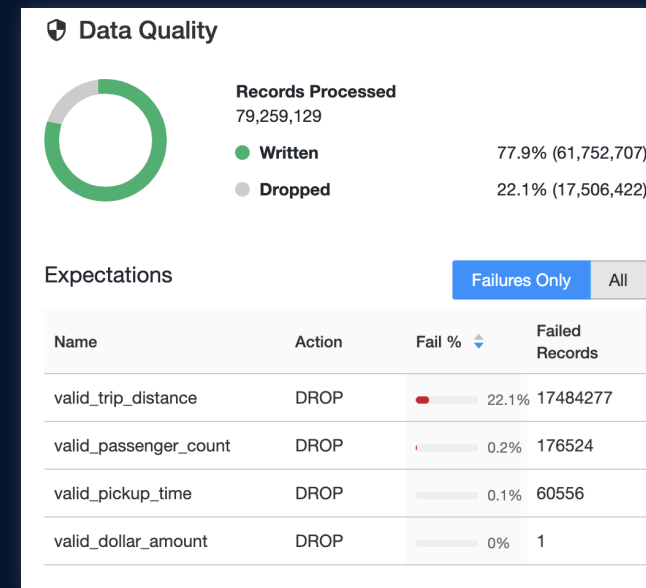
Automatically checks for errors, missing dependencies and syntax errors

Data quality and monitoring for continuous data

Ensure **correctness** with Expectations

- Define data quality and integrity controls within the pipeline with data expectations
- Address data quality errors with flexible policies: fail, drop, alert, quarantine
- All data pipeline runs and quality metrics are captured, tracked and reported

```
/* Stage 1: Bronze Table drop invalid rows */  
CREATE STREAMING TABLE fire_account_bronze AS  
( CONSTRAINT valid_account_open_dt EXPECT (acconut_dt is not  
null and (account_close_dt > account_open_dt)) ON VIOLATION DROP  
ROW  
COMMENT "Bronze table with valid account ids"  
SELECT * FROM fire_account_raw ...
```

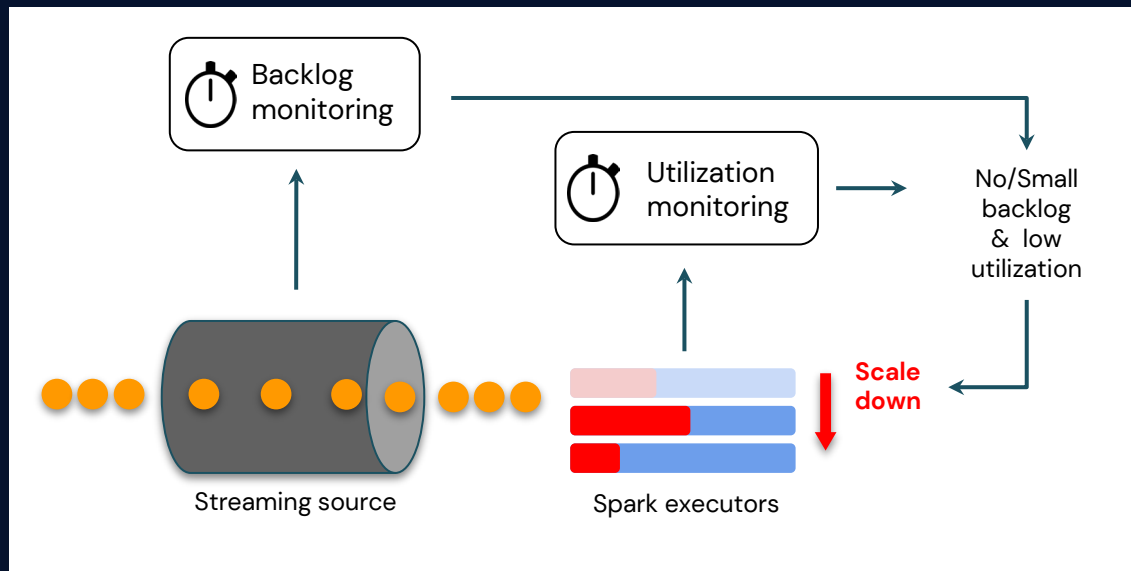


Enhanced Autoscaling for streaming workloads

Save infrastructure costs while maintaining end-to-end latency SLAs for streaming workloads

Problem

Optimize infrastructure spend when making scaling decisions for streaming workloads



- Built to handle streaming workloads which are spiky and unpredictable
- Shuts down nodes when utilization is low while guaranteeing task execution
- Only scales up to needed # of nodes

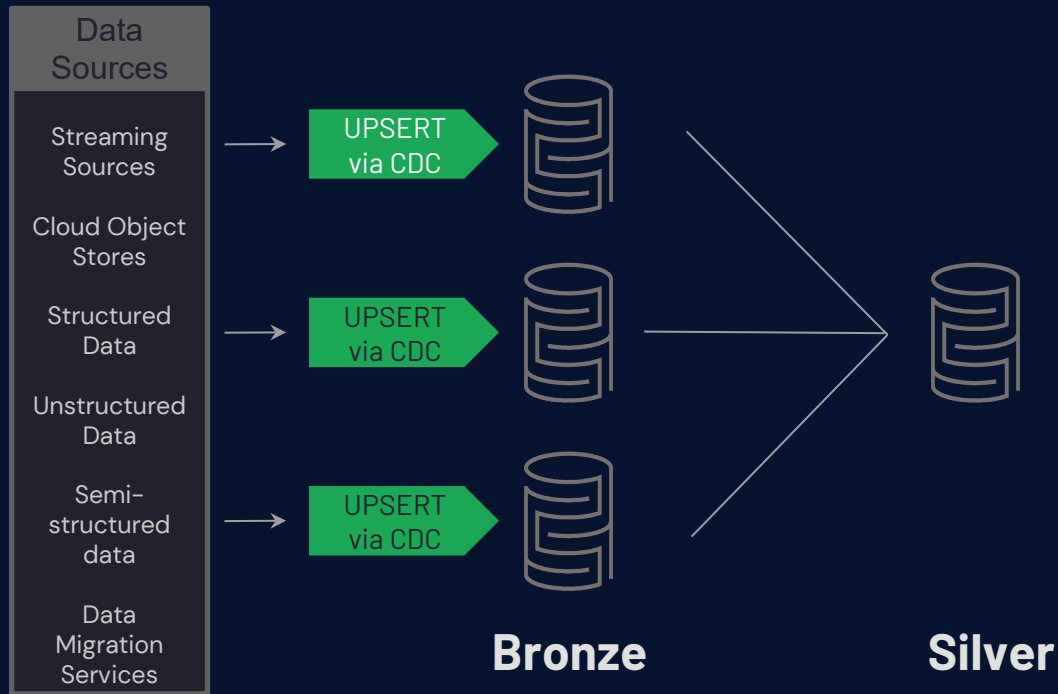
Automated streaming ETL operations

Automatically optimizes data for performance & ease of use

The screenshot displays the Databricks Pipelines interface. At the top, it shows the breadcrumb 'Pipelines / Pipeline Details' and a 'Preview' tab. The pipeline is titled 'A simple SQL Pipeline' and has an ID of '44f8b791-56ab-45e0-9f34-77fa054472c5'. Its status is 'Idle'. Action buttons include 'Delete', 'Edit Settings', 'Start', and 'Full Refresh'. Below the title, a dropdown menu is open, showing a list of pipeline runs with their completion times and statuses: 'Completed', 'Canceled', and 'Failed'. The background shows a DAG (Directed Acyclic Graph) of the pipeline's tasks, including 'tracked_data', 'all_global_configs', 'all_specified_configs', 'pipelines_tests', 'production_pipeline', 'updates', 'usage_by_region', 'usage_daily', and 'usage_by_partition'.

- Reduce down time with automatic error handling and easy replay
- Eliminate maintenance with automatic optimizations of all Delta Live Tables
- Auto-scaling adds more resources automatically when needed.

Change data capture (CDC)



- Stream change records (inserts, updates, deletes) from any data source supported by DBR, cloud storage, or DBFS
- Simple, declarative “APPLY CHANGES INTO” API for SQL or Python
- Handles out-of-order events
- Schema evolution
- SCD2 support

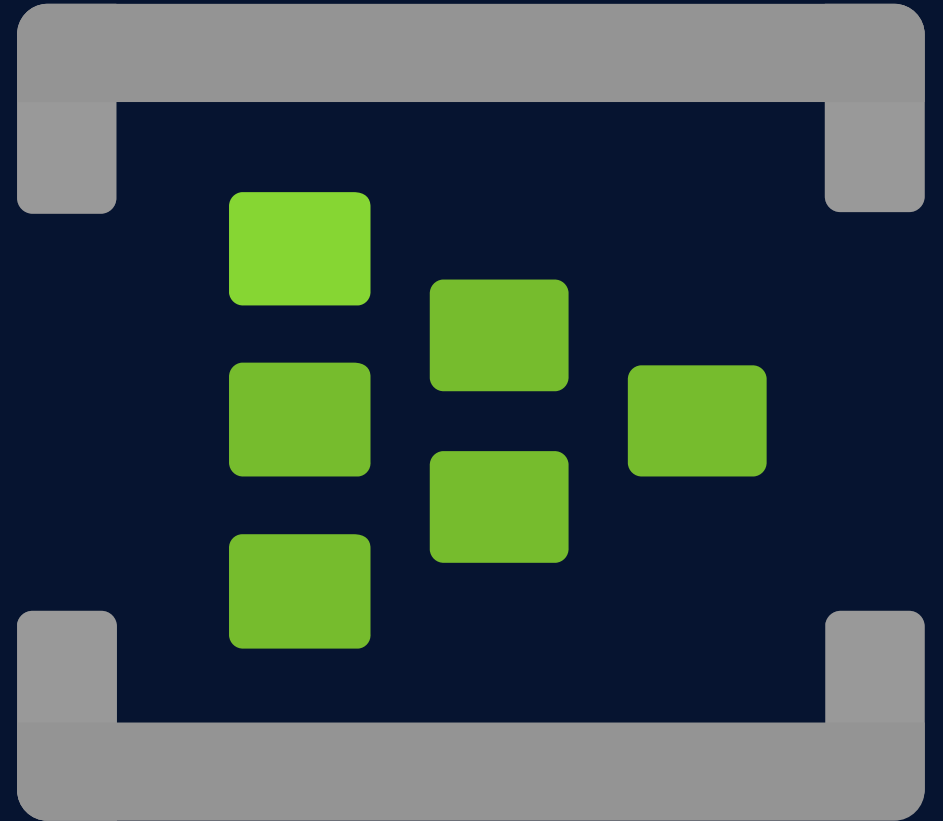
Demo

Simplify
code with
Declarative
ETL – SQL
or Python



AZURE EVENT HUBS

- Scalable message broker
- Decouple producer and consumer
- Apache Kafka compatibility
- Auto-scale namespace (TUs)

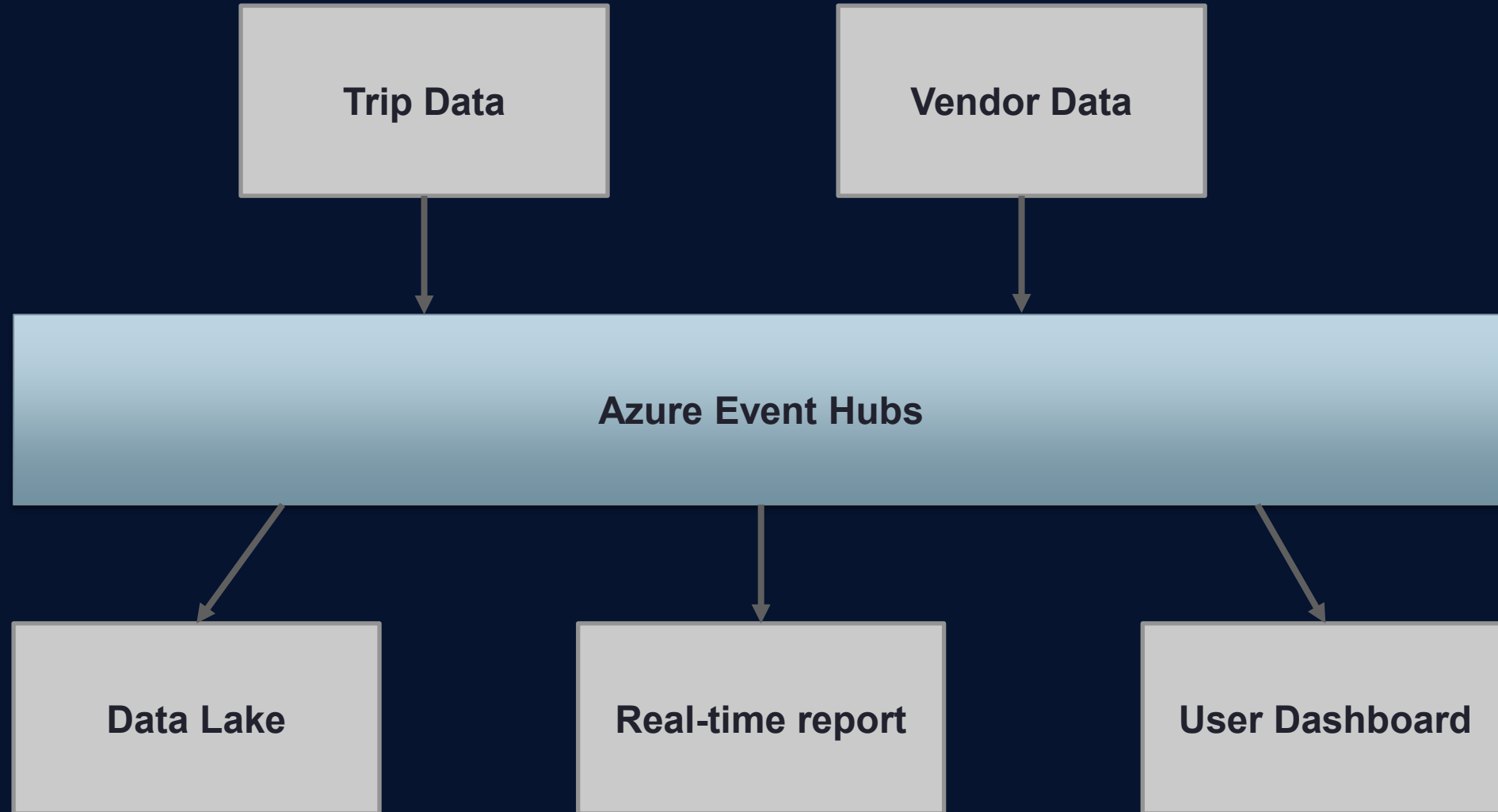


Why Event Hubs?

Reliable place to stream events;
decoupled from destination

Event Hubs is a scalable message broker, keeping up with producer and persisting data for all consumers

Hub for streaming data



Spark Streaming Benefits

- Re-use Spark batch code
- Stateful streaming and joins
- Mature with many integrations
- Kafka or Event Hubs not required

Continuous job not required

Streaming API can load incremental data in batches, such as daily or hourly

Lower cost by not using an always-on cluster

Simpler approach for aggregations and updates

Additional Training

- <https://www.databricks.com/learn>
- <https://learn.microsoft.com/en-us/training/paths/data-engineer-azure-databricks/>
- <https://www.databricks.com/discover/pages/getting-started-with-delta-live-tables>
- <https://learn.microsoft.com/en-us/training/paths/get-started-data-engineering/>
- [Design a Data Warehouse Schema](#)

Community content to help you succeed in data roles

My content

- [youtube.com/DustinVannoy](https://www.youtube.com/DustinVannoy)
- dustinvannoy.com
- <https://dustinvannoy.com/2020/04/26/journey-of-a-data-engineer-part-2/>
- [Data Engineering San Diego meetup](#)

Content by others

- [youtube.com/@AdvancingAnalytics](https://www.youtube.com/@AdvancingAnalytics)
- [youtube.com/@Ike_Ellis](https://www.youtube.com/@Ike_Ellis)
- [youtube.com/@ftdacademy](https://www.youtube.com/@ftdacademy)